

Cloud App - [ATHSNAC— UMKM E-Commerce Platform]

Deskripsi Proyek

ATHSNAC (E-Commerce UMKM RAZ'Q) adalah platform e-commerce berbasis website yang dirancang untuk mendigitalisasikan proses bisnis UMKM RAZ'Q Balikpapan. UMKM ini memproduksi makanan khas Balikpapan seperti Amplang, keripik pisang, abon, dan camilan lainnya. Aplikasi ini mengatasi kendala pemasaran dan visibilitas usaha, serta mempermudah pengelolaan transaksi dan stok produk yang sebelumnya menghadapi tantangan kompleksitas pada aplikasi pihak ketiga.

Daftar Isi

-  Cloud App - [ATHSNAC— UMKM E-Commerce Platform]
 -  Deskripsi Proyek
 -  Daftar Isi
 - Fitur Utama
 -  Manajemen Produk & Katalog
 -  Keranjang Belanja
 -  Manajemen Pesanan
 -  Pembayaran
 -  Testimoni & Ulasan
 -  Sistem & Keamanan
 - Fitur Berdasarkan Role
 -  Tim
 -  Tech Stack
 -  Arsitektur Sistem
 - **Diagram 1: Overall System Architecture**
 - **Diagram 2: Frontend Component & Service Architecture**
 - **Diagram 3: Backend API Endpoints by Module**
 - **Diagram 4: Entity Relationship Diagram (ERD)**
 -  Getting Started
 - Prasyarat
 -  Cara Menjalankan
 - 1. Clone repository
 - 2. Jalankan Backend
 - 4. Jalankan Frontend
 - 5. Verifikasi
 -  Docker & Docker Compose
 - Quick Start dengan Docker Compose
 - Docker Compose Commands
 - Docker Hub Image Reference
 - Environment Setup untuk Docker
 -  Struktur Proyek
 -  Dasar Teori
 - 1. API (Application Programming Interface)

- 2. REST (Representational State Transfer)
- 3. HTTP Methods & CRUD
- 4. HTTP Status Codes
- 5. Database Relasional & PostgreSQL
- 6. ORM — SQLAlchemy
- 7. Pydantic — Validasi Data
- 8. FastAPI
-  Panduan Membangun REST API
-  Dasar Teori
 - 1. API (Application Programming Interface)
 - 2. REST (Representational State Transfer)
 - 3. HTTP Methods & CRUD
 - 4. HTTP Status Codes
 - 5. Database Relasional & PostgreSQL
 - 6. ORM — SQLAlchemy
 - 7. Pydantic — Validasi Data
 - 8. FastAPI
- | **Type hints** | Memanfaatkan type hints Python untuk validasi dan dokumentasi |
-  Panduan Membangun Frontend React
-  Dasar Teori
 - 1. React
 - 2. Props dan State
 - 3. Fetch API
 - 4. Separation of Concerns pada Frontend
-  Fitur UI yang Dibangun
-  Dokumentasi API
 - Base URL
-  Tabel Ringkasan Endpoint
 - Alur Autentikasi
 - Contoh Response Login
-  Authentication
 - Konsep JWT
 - 1. **Register (Daftar Akun Baru)**
 - 2. **Login (Dapatkan Token)**
 - 3. **Get Current User (Verifikasi Token)**
 - 4. **Cara Menggunakan Token di Request**
 - 5. **Role-Based Access Control (RBAC)**
 - 6. **Token Security Best Practices**
-  Dokumentasi
 -  Quality Assurance & Testing
 -  Development Documentation
- | [docs/api-documentation.md](#) | Backend | Dokumentasi lengkap API endpoints
-  Roadmap

Fitur Utama

 Manajemen Produk & Katalog

- ☒ **Katalog Produk** — Daftar produk lengkap dengan gambar, harga, kategori, dan stok real-time
- ☒ **Filter & Pencarian** — Cari produk berdasarkan nama, deskripsi, atau kategori
- ☒ **Statistik Inventori** — Dashboard admin dengan total produk, total nilai stok, dan breakdown per kategori
- ☒ **CRUD Produk** — Admin dapat menambah, mengubah, dan menghapus produk

Keranjang Belanja

- ☒ **Keranjang Aktif** — Setiap pelanggan memiliki keranjang belanja yang persisten
- ☒ **Snapshot Harga** — Harga produk saat ditambahkan ke keranjang tersimpan otomatis
- ☒ **Update Quantity** — Ubah jumlah item tanpa menghapus dari keranjang

Manajemen Pesanan

- ☒ **Pemesanan Online** — Buat pesanan langsung dari aplikasi dengan kode order unik
- ☒ **Riwayat Pesanan** — Pelanggan dapat melihat semua pesanan mereka
- ☒ **Manajemen Status** — Admin dapat mengubah status pesanan dari **pending** hingga **delivered**
- ☒ **Dashboard Admin** — Admin dapat melihat semua pesanan dari seluruh pelanggan

Pembayaran

- ☒ **Multi-Metode** — Mendukung **credit_card**, **bank_transfer**, **e_wallet**, **cash**
- ☒ **Upload Bukti** — Pelanggan dapat mengunggah URL bukti pembayaran
- ☒ **Verifikasi Admin** — Admin memverifikasi pembayaran dan mengubah status
- ☒ **Riwayat Pembayaran** — Tersedia untuk pelanggan dan admin

★ Testimoni & Ulasan

- ☒ **Rating & Komentar** — Pelanggan dapat memberikan rating bintang 1-5 dan komentar
- ☒ **Verifikasi Pembelian** — Testimoni terhubung ke order untuk membuktikan pembelian nyata
- ☒ **Moderasi Admin** — Admin dapat menyembunyikan/menampilkan testimoni
- ☒ **Filter Publik** — Hanya testimoni yang **is_visible = true** yang tampil ke publik

Sistem & Keamanan

- ☒ **JWT Authentication** — Login & registrasi dengan token berbatas waktu (60 menit)
- ☒ **Role-Based Access** — Pembatasan akses berdasarkan role **customer** dan **admin**
- ☒ **Password Hashing** — Password disimpan sebagai hash bcrypt, tidak pernah plain-text

Fitur Berdasarkan Role

Pelanggan

- ☒ Melihat profil, kontak, dan katalog produk UMKM
- ☒ Melakukan registrasi akun dan login ke sistem
- ☒ Menambahkan barang ke keranjang dan melakukan pemesanan produk
- ☒ Melakukan pembayaran via QRIS dan mengunggah bukti transaksi
- ☒ Menambahkan testimoni atau ulasan produk

Admin

- ☒ Login ke Dashboard admin dengan hak akses kontrol penuh
- ☒ Mengelola data akun admin dan melihat data pelanggan
- ☒ Menambah, mengubah, dan menghapus produk di katalog
- ☒ Memantau stok barang dan mendapatkan notifikasi jika stok hampir habis
- ☒ Memverifikasi pembayaran dan mengubah status pembelian pelanggan

Tim

Nama	NIM	Peran
Andini Permata Dewanti	10231014	Lead Backend
Putri Rahmawati	10231074	Lead Frontend
Krishandy Dhanyssa Pratama	10231050	Lead DevOps
Desnita Dwi Putri	10231030	Lead QA & Docs

Tech Stack

Teknologi	Fungsi	Penjelasan
FastAPI	Backend REST API	Framework Python berbasis asynchronous untuk membangun REST API dengan performa tinggi dan dokumentasi otomatis via Swagger UI.
Uvicorn	Server	ASGI server untuk menjalankan FastAPI.
SQLAlchemy	ORM	Menerjemahkan Python object ke SQL dan sebaliknya, sehingga tidak perlu menulis query SQL secara manual.
Pydantic v2	Validasi Data	Memvalidasi format data request dan response serta mendefinisikan schema API.
python-jose	JWT	Membuat dan memverifikasi JWT token untuk sistem autentikasi.
passlib + bcrypt	Keamanan Password	Mengubah password menjadi hash bcrypt sebelum disimpan ke database, sehingga password asli tidak pernah tersimpan.
python-dotenv	Konfigurasi	Membaca variabel sensitif (password, secret key) dari file <code>.env</code> agar tidak ter-commit ke Git.
React 18	Frontend	Library JavaScript untuk membangun antarmuka pengguna berbasis komponen yang responsif dan modular.
Vite	Build Tool	Development server dan build tool untuk frontend React dengan hot-reload cepat. Berjalan di port 5173.
Fetch API	HTTP Client	API bawaan JavaScript untuk mengirim request HTTP (GET/POST/PUT/DELETE) dari frontend ke backend.
PostgreSQL	Database	Sistem manajemen basis data relasional untuk menyimpan data user dan item inventori secara terstruktur.

Teknologi	Fungsi	Penjelasan
Docker	Containerization	Mengemas aplikasi beserta seluruh dependensinya ke dalam container agar environment berjalan konsisten di berbagai sistem.
GitHub Actions	CI/CD	Mengotomatisasi proses build, testing, dan deployment setiap ada perubahan yang di-push ke repository.
Railway / Render	Cloud Deployment	Platform cloud untuk hosting backend dan database agar aplikasi dapat diakses secara online.

Arsitektur Sistem

Proyek ATHSNAC menggunakan arsitektur **three-tier** yang memisahkan presentation layer (Frontend), business logic layer (Backend), dan data layer (Database) secara bersih.

Diagram 1: Overall System Architecture

```
graph TB
    subgraph "🖥 Frontend Layer (React + Vite)"
        Customer["👤 Customer Interface<br/>- CustomerHome<br/>- ShopPage<br/>- CartPage<br/>- OrdersPage"]
        Admin["🔑 Admin Dashboard<br/>- AdminDashboard<br/>- AdminProducts<br/>- AdminOrders<br/>- AdminPayments"]
        Auth["🔐 Auth Pages<br/>- LoginPage<br/>- RegisterPage<br/>- ProtectedRoute"]
    end

    subgraph "⚙ Backend Layer (FastAPI + Python)"
        Routes["🗺 Routes/Endpoints<br/>main.py"]
        Schemas["📄 Data Validation<br/>schemas.py<br/>Pydantic"]
        Auth_Service["🔑 Auth Service<br/>auth.py<br/>JWT + bcrypt"]
        CRUD["🔄 Business Logic<br/>crud.py<br/>CRUD Ops"]
    end

    subgraph "🗄 Data Layer (PostgreSQL)"
        Models["📊 SQLAlchemy Models<br/>models.py<br/>ORM"]
        Database["📋 PostgreSQL Database<br/>Users | Products | Orders<br/>Carts | Payments | Testimonials"]
    end

    subgraph "🔌 Services"
        API_Service["API Service Layer<br/>services/api.js<br/>Fetch Wrapper"]
    end

    Customer -->|HTTP GET/POST/PUT/DELETE| API_Service
    Admin -->|HTTP with JWT Token| API_Service
    Auth -->|Auth Requests| API_Service

    API_Service -->|REST API Calls| Routes
    Routes -->|Validate Input| Schemas
    Routes -->|Auth Check| Auth_Service
```

```

Routes -->|Execute Logic| CRUD

CRUD -->|ORM Queries| Models
Models -->|SQL| Database

Database -->|Result Set| Models
Models -->|Python Objects| CRUD
CRUD -->|JSON Response| Routes
Routes -->|HTTP 200/201/400/401| API_Service
API_Service -->|Update State| Customer
API_Service -->|Update State| Admin

style Customer fill:#e1f5e1
style Admin fill:#ffe1e1
style Auth fill:#e1e5ff
style Routes fill:#fff4e1
style Schemas fill:#ffe1f5
style Auth_Service fill:#e1fff5
style CRUD fill:#f5e1ff
style Models fill:#ffe1e1
style Database fill:#e1e1ff
style API_Service fill:#fff9e1

```

Diagram 2: Frontend Component & Service Architecture

```

graph TD
    App["🏠 App.jsx<br/>Root + Routing + Auth State"]
    
    subgraph Auth_Pages["🔒 Auth Pages"]
        Login["LoginChoicePage<br/>LoginPage<br/>RegisterPage"]
    end
    
    subgraph Customer_Pages["👤 Customer Pages"]
        CustHome["CustomerHome"]
        Shop["ShopPage"]
        Detail["ProductDetailPage"]
        Cart["CartPage"]
        Checkout["CheckoutPage"]
        Orders["OrdersPage"]
        Profile["ProfilePage"]
        Testim["TestimoniPage"]
    end
    
    subgraph Admin_Pages["🔒 Admin Pages"]
        AdminDash["AdminDashboard"]
        AdminProd["AdminProducts"]
        AdminOrd["AdminOrders"]
        AdminPay["AdminPayments"]
        AdminCust["AdminCustomers"]
        AdminTest["AdminTestimonials"]
    end

```

```

subgraph Components["🧩 Reusable Components"]
  Header["Header.jsx"]
  Nav["CustomerNav.jsx"]
  ItemCard["ItemCard.jsx"]
  ItemForm["ItemForm.jsx"]
  ItemList["ItemList.jsx"]
  SearchBar["SearchBar.jsx"]
end

subgraph Services["🔌 Services"]
  ApiService["services/api.js<br/>- login(), register()<br/>- getProducts(),
createProduct()<br/>- getCart(), addToCart()<br/>- createOrder(), getOrders()
<br/>- uploadPayment(), verifyPayment()<br/>- submitTestimony()"]
end

App -->|renders| Auth_Pages
App -->|renders + Header + Nav| Customer_Pages
App -->|renders + Header| Admin_Pages

Customer_Pages -->|uses| Components
Admin_Pages -->|uses| Components

Components -->|calls| ApiService
Auth_Pages -->|calls| ApiService

ApiService -->|HTTP/JWT| Backend["🌀 FastAPI Backend<br/>:8000"]
Backend -->|REST API| Database["💾 PostgreSQL<br/>:5432"]

style App fill:#fff9e1
style Auth_Pages fill:#e1e5ff
style Customer_Pages fill:#e1f5e1
style Admin_Pages fill:#ffe1e1
style Components fill:#f5e1ff
style Services fill:#ffe1f5
style Backend fill:#e1fff5
style Database fill:#e1e1ff

```

Diagram 3: Backend API Endpoints by Module

```

graph TD
  API["FastAPI Backend<br/>:8000"]

  subgraph Auth["🔑 Authentication"]
    Login["POST /auth/login"]
    Register["POST /auth/register"]
    GetMe["GET /auth/me"]
  end

  subgraph Products["📦 Products Module"]
    ListProd["GET /products"]
    CreateProd["POST /products<br/>Admin Only"]
    DetailProd["GET /products/{id}"]
  end

```

```

    UpdateProd["PUT /products/{id}<br/>Admin Only"]
    DeleteProd["DELETE /products/{id}<br/>Admin Only"]
    StatsProd["GET /products/stats<br/>Admin Only"]
end

subgraph Cart["🛒 Shopping Cart"]
    ViewCart["GET /cart"]
    AddCart["POST /cart/items"]
    UpdateCart["PUT /cart/items/{id}"]
    DeleteCart["DELETE /cart/items/{id}"]
end

subgraph Orders["📦 Orders"]
    CreateOrder["POST /orders"]
    ListOrder["GET /orders"]
    AdminOrders["GET /orders/admin/all<br/>Admin Only"]
    UpdateOrder["PUT /orders/{id}<br/>Admin Only"]
end

subgraph Payments["💵 Payments"]
    CreatePay["POST /payments"]
    ListPay["GET /payments"]
    VerifyPay["PUT /payments/{id}<br/>Admin Only"]
end

subgraph Testimonials["★ Testimonials"]
    CreateTest["POST /testimonials"]
    ListTest["GET /testimonials"]
    UpdateTest["PUT /testimonials/{id}"]
    ToggleTest["PUT /testimonials/{id}/toggle-visibility<br/>Admin Only"]
end

API --> Auth
API --> Products
API --> Cart
API --> Orders
API --> Payments
API --> Testimonials

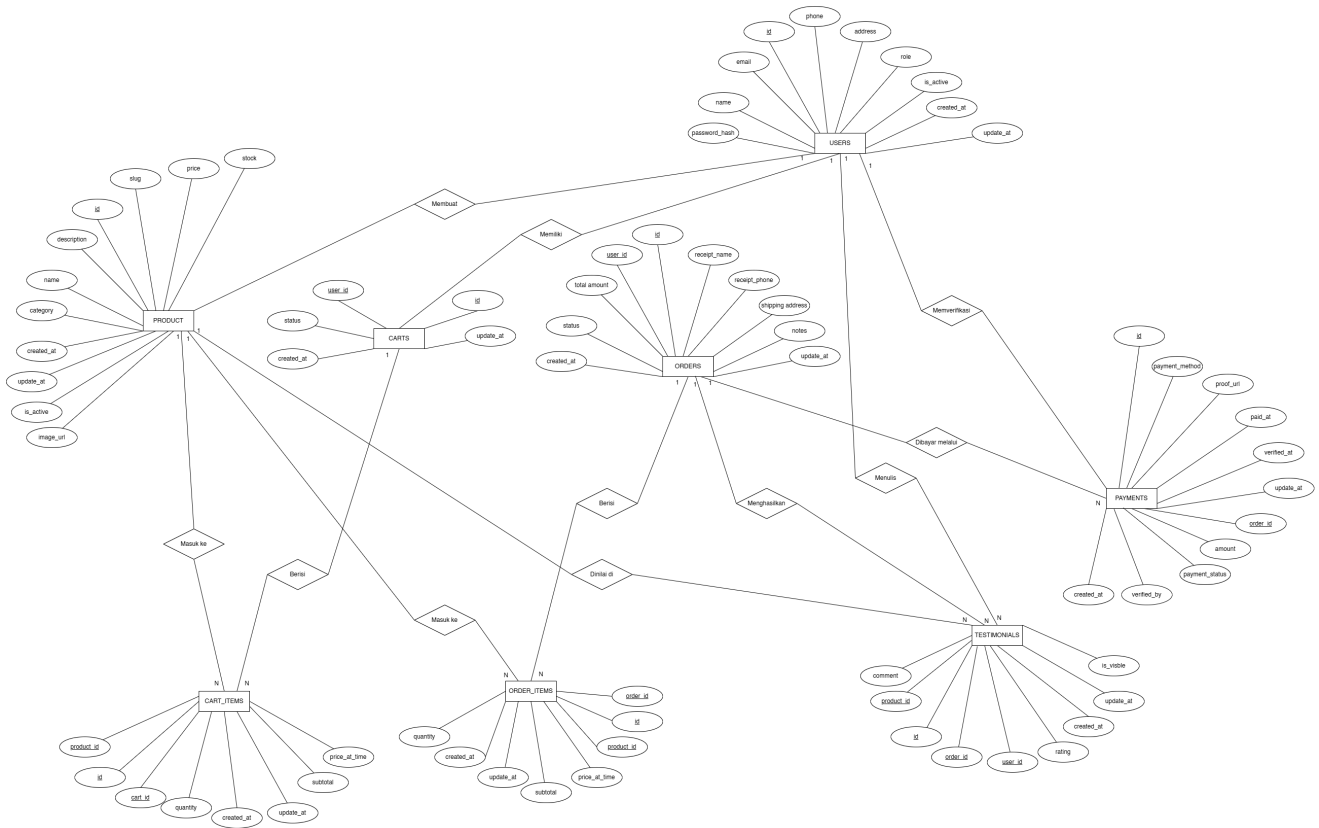
Auth -->|validate & create token| DB[(PostgreSQL<br/>users)]
Products -->|CRUD operations| DB[(PostgreSQL<br/>products)]
Cart -->|store cart| DB[(PostgreSQL<br/>cart_items)]
Orders -->|track orders| DB[(PostgreSQL<br/>orders)]
Payments -->|validate payments| DB[(PostgreSQL<br/>payments)]
Testimonials -->|manage reviews| DB[(PostgreSQL<br/>testimonials)]

style API fill:#fff9e1
style Auth fill:#e1e5ff
style Products fill:#e1f5e1
style Cart fill:#fff4e1
style Orders fill:#ffe1f5
style Payments fill:#e1fff5
style Testimonials fill:#ffe1e1
style DB fill:#e1e1ff

```


- **Frontend (React + Vite)** berjalan di port **5173** dengan hot-reload development server
- **Backend (FastAPI + Uvicorn)** berjalan di port **8000** dengan auto-documentation Swagger UI
- **Database (PostgreSQL)** di port **5432** dengan SQLAlchemy ORM untuk abstraksi query
- **JWT Authentication** memastikan setiap request authenticated dan ter-validasi role-nya
- **Separation of Concerns** memisahkan routing, validasi, business logic, dan data access dalam file terpisah

Diagram 4: Entity Relationship Diagram (ERD)



Tabel	Deskripsi	Field Utama
USER	Menyimpan data pengguna (customer & admin)	id, email, name, password_hash, role, phone, address
PRODUCT	Katalog produk UMKM	id, name, description, category, price, stock, image_url
CART	Keranjang belanja per user	id, user_id, status
CARTITEM	Item dalam keranjang	id, cart_id, product_id, quantity, price_at_time, subtotal
ORDER	Pesanan yang ditempatkan customer	id, user_id, order_code, receipt_name, total_amount, status

Tabel	Deskripsi	Field Utama
ORDERITEM	Item dalam order	id, order_id, product_id, quantity, price_at_time, subtotal
PAYMENT	Metode pembayaran & verifikasi	id, order_id, payment_method, amount, payment_status, verified_by
TESTIMONIAL	Review/rating produk dari customer	id, product_id, user_id, order_id, rating, comment, is_visible

Hubungan Antar Tabel:

Relasi	Penjelasan
USER → CART	Setiap pelanggan punya 1 keranjang belanja (bisa aktif atau tidak)
USER → ORDER	Setiap pelanggan bisa membuat banyak pesanan
USER → TESTIMONIAL	Setiap pelanggan bisa menulis banyak review/rating produk
USER → PAYMENT	Admin bisa memverifikasi banyak pembayaran dari customer
CART → CARTITEM	Setiap keranjang berisi banyak item produk
CARTITEM → PRODUCT	Setiap item keranjang terhubung ke 1 produk tertentu
ORDER → ORDERITEM	Setiap pesanan berisi banyak item produk
ORDERITEM → PRODUCT	Setiap item pesanan terhubung ke 1 produk tertentu
ORDER → PAYMENT	Setiap pesanan bisa memiliki banyak pembayaran (untuk cicilan)
ORDER → TESTIMONIAL	Setiap pesanan bisa punya banyak review dari produk berbeda
PRODUCT → TESTIMONIAL	Setiap produk bisa menerima banyak review dari customer

Fitur Penting:

- **Auto Hapus Terkait** — Jika customer dihapus, semua keranjang, pesanan, review, dan pembayaran-nya otomatis terhapus juga
- **Simpan Harga** — Harga produk saat ditambah ke keranjang/pesanan disimpan tetap, tidak ikut berubah jika harga produk berubah
- **Track Status** — Pesanan dan pembayaran punya status (pending, success, dll) untuk memantau progres
- **Verifikasi Pembelian** — Review harus terhubung ke pesanan asli agar tidak ada review palsu dari yang tidak beli

Getting Started

Prasyarat

Wajib Diinstall:

- **Docker & Docker Compose** (Opsional, untuk development dengan container)
 - Download dari docker.com

- Verifikasi: `docker --version` dan `docker compose --version`
- **Python 3.10+**: Diperlukan untuk menjalankan modul FastAPI, SQLAlchemy, dan async logic
- **PostgreSQL 12+**: ⚠ **WAJIB** — Backend menggunakan database PostgreSQL untuk menyimpan data (users, products, orders, payments, testimonials)
 - Di Windows: Download dari [postgresql.org](https://www.postgresql.org)
 - Di macOS: `brew install postgresql`
 - Di Linux: `sudo apt-get install postgresql`
 - **Pastikan PostgreSQL sudah berjalan** sebelum menjalankan backend
 - Catatan: User yang digunakan di `.env` adalah `postgres` dengan port default `5432`
- **Node.js 18+ (include npm)**: Diperlukan untuk build dan dependency management frontend React
- **Git**: Untuk manajemen versi dan kolaborasi tim

i Verifikasi Instalasi:

```
# Check Python version
python --version

# Check PostgreSQL version & status
psql --version
psql -U postgres -c "SELECT version();"

# Check Node.js & npm version
node --version
npm --version

# Check Git
git --version
```

Cara Menjalankan

⚠ **Perlu 3 terminal berjalan** — terminal PostgreSQL (jika perlu), backend, dan frontend.

1. Clone repository

```
git clone https://github.com/aidilsaputrakirsan-classroom/cloud-team-ignite.git
cd cloud-team-ignite
```

2. Jalankan Backend

Buka terminal pertama untuk backend:

```
cd backend

# Copy environment file
cp .env.example .env

# Edit .env — isi konfigurasi PostgreSQL
```

```
#  
DATABASE_URL=postgresql://athsnack_user:your_secure_password@localhost:5432/athsnack  
# SECRET_KEY=<generate dengan: openssl rand -hex 32>  
  
# Install dependencies  
pip install -r requirements.txt  
  
# Jalankan backend  
uvicorn main:app --reload --port 8000
```

- ☑ Backend tersedia di <http://localhost:8000>
- 📖 Dokumentasi API (Swagger UI): <http://localhost:8000/docs>

4. Jalankan Frontend

Buka terminal **baru** (jangan tutup terminal backend):

```
cd frontend  
  
# Copy environment file  
cp .env.example .env  
  
# Pastikan .env berisi: VITE_API_URL=http://localhost:8000  
cat .env  
  
# Install dependencies  
npm install  
  
# Jalankan development server  
npm run dev
```

- ☑ Frontend tersedia di <http://localhost:5173>

5. Verifikasi

Buka <http://localhost:5173> — halaman **Login** akan tampil. Lakukan:

1. Registrasi akun baru (role: Customer)
2. Login dengan email & password yang baru didaftar
3. Cek apakah data tersimpan di database (akses Swagger UI di <http://localhost:8000/docs>)
4. Untuk login sebagai Admin, hubungi team lead atau gunakan admin seed data (jika ada)

Docker & Docker Compose

Quick Start dengan Docker Compose

Alternatif **mudah** tanpa perlu setup PostgreSQL, backend, dan frontend secara terpisah:

```
# Clone repository
git clone https://github.com/aidilsaputrakirsan-classroom/cloud-team-ignite.git
cd cloud-team-ignite

# Jalankan semua services (backend, frontend, postgres)
docker compose up -d

# Tunggu ~10 detik untuk services startup
sleep 10

# Verifikasi semua services berjalan
docker compose ps
```

☒ Aplikasi siap:

- **Frontend:** http://localhost:5173
- **Backend API:** http://localhost:8000/docs
- **PostgreSQL:** localhost:5432

Docker Compose Commands

```
# Lihat status semua services
docker compose ps

# Lihat logs real-time
docker compose logs -f

# Lihat logs service tertentu
docker compose logs -f backend
docker compose logs -f frontend
docker compose logs -f db

# Jalankan services di background
docker compose up -d

# Stop semua services (data tetap tersimpan)
docker compose down

# Stop services & hapus semua data/volumes
docker compose down -v

# Restart semua services
docker compose restart

# Rebuild images & jalankan
docker compose up -d --build
```

Docker Hub Image Reference

Images yang digunakan:

Service	Image	Version	Purpose
Backend	python:3.11-slim	3.11	FastAPI + SQLAlchemy
Frontend	node:18-alpine	18	React + Vite build
Database	postgres:15-alpine	15	PostgreSQL dengan Alpine
Nginx	nginx:alpine	Alpine	Reverse proxy & static serve

Custom Images (dari Dockerfile lokal):

- `athsnac-backend:latest` — dibangun dari `backend/Dockerfile`
- `athsnac-frontend:latest` — dibangun dari `frontend/Dockerfile`

Environment Setup untuk Docker

File `.env` di root project (jika menggunakan docker-compose):

```
# Database
POSTGRES_USER=athsnac_user
POSTGRES_PASSWORD=your_secure_password_here
POSTGRES_DB=athsnac_db
DATABASE_URL=postgresql://athsnac_user:your_secure_password_here@db:5432/athsnac_db

# Backend
SECRET_KEY=your_super_secret_key_generate_with_openssl_rand_hex_32
DEBUG=False

# Frontend (di dalam container sudah otomatis menggunakan backend:8000)
VITE_API_URL=http://localhost:8000
```

📁 Struktur Proyek

```
cloud-team-ignite/
├── backend/
│   ├── main.py           ← Entry point, router, CORS, semua endpoint
│   ├── auth.py           ← JWT utilities: buat token, verifikasi, hash
│   └── password
│       ├── database.py   ← Koneksi PostgreSQL via SQLAlchemy
│       ├── models.py     ← SQLAlchemy models: tabel items & users
│       └── schemas.py    ← Pydantic schemas: validasi request/response +
├── auth
│   ├── crud.py           ← Fungsi CRUD items & user (create,
│   └── authenticate)
│       ├── requirements.txt ← Daftar dependencies Python
│       ├── .env           ← 🚫 RAHASIA – berisi password & secret, jangan
├── di-commit!
│   ├── .env.example      ← ☑️ Template konfigurasi – ini yang di-commit
└── frontend/
```

					src/	
						App.jsx
						← Root component & routing dengan React Router
						main.jsx
						← Entry point React, render App component
						App.css
						← Global styling & CSS variables
						index.css
						← CSS reset & utility styles
						components/
						LoginChoicePage.jsx
						← Halaman pilih jenis login
						(Customer/Admin)
						LoginPage.jsx
						← Form login untuk customer & admin
						RegisterPage.jsx
						← Form registrasi akun baru
						ProtectedRoute.jsx
						← Route guard untuk autentikasi & role-based access
						Header.jsx
						← Header navigasi, judul app, cart icon, profile menu
						CustomerNav.jsx
						← Navigation sidebar/topnav khusus customer
						ItemForm.jsx
						← Form CRUD produk (create & edit)
						ItemList.jsx
						← Grid container daftar produk dengan loading/empty state
						ItemCard.jsx
						← Card produk: foto, nama, harga, stok, tombol aksi
						SearchBar.jsx
						← Search input & filter kategori
						pages/
						[Customer Pages]
						CustomerHome.jsx
						← Beranda customer dengan hero & featured products
						ShopPage.jsx
						← Katalog produk dengan filter & search
						ProductDetailPage.jsx
						← Detail produk, stok, rating, testimoni, add to cart
						CartPage.jsx
						← Keranjang belanja, update qty, subtotal
						CheckoutPage.jsx
						← Form checkout & pengiriman, buat order
						OrdersPage.jsx
						← Riwayat pesanan customer dengan tracking status
						TestimoniPage.jsx
						← Daftar testimoni publik dari customer
						ProfilePage.jsx
						← Profil customer, edit data, riwayat transaksi
						AboutPage.jsx
						← Informasi tentang ATHSNAC
						[Admin Pages]
						AdminDashboard.jsx
						← Dashboard utama admin dengan KPI & statistics
						AdminProducts.jsx
						← Manajemen produk CRUD & update stok
						AdminOrders.jsx
						← Manajemen pesanan & update status pengiriman
						AdminPayments.jsx
						← Verifikasi pembayaran dari customer
						AdminCustomers.jsx
						← Data pelanggan & history transaksi
						AdminTestimonials.jsx
						← Moderasi testimoni (approve/reject/hide)
						services/
						api.js
						← Service layer: fetch wrapper, HTTP methods, token management
						public/
						← Static assets (favicon, logo, dll)
						.env
						←  RAHASIA – berisi VITE_API_URL & env variables
						.env.example
						← ☒ Template .env – ini yang di-commit
						eslint.config.js
						← ESLint configuration untuk code quality

├── index.html	← HTML entry point
├── package.json	← Dependencies & scripts Node.js
├── vite.config.js	← Vite bundler configuration
└── README.md	← Dokumentasi setup frontend
├── backend/	
│ ├── Dockerfile	← Konfigurasi Docker image backend
│ ├── .dockerignore	← Daftar file yang tidak masuk ke Docker image
│ └── ...	
├── docs/	
│ ├── api-test-results.md	← Dokumentasi hasil testing endpoint API
│ ├── ui-test-results.md	← Dokumentasi hasil testing UI React
│ ├── auth-test-results.md	← Dokumentasi hasil testing autentikasi JWT
│ ├── image-comparison.md	← Perbandingan ukuran Docker image
│ ├── docker-cheatsheet.md	← Referensi perintah Docker
│ ├── database-schema.md	← Skema tabel database PostgreSQL
│ └── member-[NAMA].md	← File verifikasi masing-masing anggota
├── .gitignore	← Daftar file yang tidak di-commit (termasuk
└── .env)	
└── README.md	← Dokumentasi proyek (file ini)

Dasar Teori

Sebelum memulai implementasi, penting untuk memahami konsep-konsep dasar yang menjadi fondasi dari proyek ini.

1. API (Application Programming Interface)

API adalah "kontrak" atau antarmuka yang mendefinisikan bagaimana dua perangkat lunak berkomunikasi satu sama lain. Dalam konteks web, API memungkinkan frontend (browser/aplikasi mobile) berbicara dengan backend (server) melalui protokol HTTP.

 **Analogi:** API seperti **pelayan di restoran**. Kamu (frontend/client) memesan makanan lewat pelayan (API), pelayan menyampaikan pesanan ke dapur (backend/server), lalu membawa makanan (response) kembali ke mejamu. Kamu tidak perlu tahu cara memasak — cukup tahu cara memesan.

2. REST (Representational State Transfer)

REST adalah gaya arsitektur desain API yang menggunakan HTTP sebagai protokol komunikasi. REST mengorganisasi data sebagai *resources* yang bisa diakses melalui URL yang konsisten dan mudah diprediksi.

Prinsip utama REST:

Prinsip	Penjelasan
Client-Server	Frontend dan backend dipisahkan dan dapat dikembangkan secara independen
Stateless	Setiap request berdiri sendiri — server tidak menyimpan informasi tentang request sebelumnya

Prinsip	Penjelasan
Uniform Interface	URL yang konsisten dan dapat diprediksi untuk setiap resource
Resource-Based	Setiap "hal" (item, user, order) adalah sebuah resource dengan URL uniknya sendiri

3. HTTP Methods & CRUD

REST API menggunakan **HTTP Methods** untuk mendefinisikan jenis operasi yang dilakukan pada sebuah resource. Setiap method berkorespondensi dengan satu operasi **CRUD**:

HTTP Method	Operasi CRUD	Contoh Endpoint	Deskripsi
GET	Read	GET /items	Ambil semua items
GET	Read	GET /items/1	Ambil item dengan id=1
POST	Create	POST /items	Buat item baru
PUT	Update	PUT /items/1	Update seluruh data item id=1
DELETE	Delete	DELETE /items/1	Hapus item id=1

4. HTTP Status Codes

Server selalu mengembalikan **status code** di setiap response untuk memberitahu client apakah request berhasil atau gagal dan mengapa.

Kode	Nama	Kapan Digunakan
200	OK	Request berhasil (GET, PUT)
201	Created	Resource baru berhasil dibuat (POST)
204	No Content	Berhasil tetapi tidak ada data dikembalikan (DELETE)
400	Bad Request	Data yang dikirim tidak valid
404	Not Found	Resource tidak ditemukan di server
422	Unprocessable Entity	Validasi gagal — format data salah (default FastAPI)
500	Internal Server Error	Terjadi kesalahan di sisi server

5. Database Relasional & PostgreSQL

Database relasional menyimpan data dalam bentuk **tabel** yang saling terhubung satu sama lain — mirip seperti spreadsheet Excel, tetapi jauh lebih powerful dan handal.

PostgreSQL dipilih dalam proyek ini karena:

- Open-source dan gratis
- Sangat *reliable* dan sudah terbukti di lingkungan produksi

- Mendukung tipe data yang kaya (JSON, Array, UUID, dll.)
- Didukung oleh hampir semua cloud provider (Railway, Render, AWS RDS, Supabase)
- Cocok untuk arsitektur microservices di fase selanjutnya

6. ORM — SQLAlchemy

ORM (Object-Relational Mapping) adalah teknik yang memungkinkan kita berinteraksi dengan database menggunakan objek Python, tanpa harus menulis SQL secara manual.

Perbandingan tanpa ORM vs dengan ORM:

Tanpa ORM (Raw SQL)	Dengan ORM (SQLAlchemy)
<code>cursor.execute("INSERT INTO items (name, price) VALUES (%s, %s)", ("Laptop", 15000000))</code>	<code>db.add(Item(name="Laptop", price=15000000))</code>
Harus menulis SQL manual	Menggunakan Python object — lebih intuitif
Rentan SQL Injection jika tidak hati-hati	Aman dari SQL Injection secara default
Tidak portable antar database	Bisa pindah database tanpa mengubah kode

Cara kerja SQLAlchemy:

```
Python Object → SQLAlchemy ORM → SQL Query → PostgreSQL
Item(name="Laptop") → translasi otomatis → INSERT INTO items... → data tersimpan
```

7. Pydantic — Validasi Data

Pydantic adalah library validasi data Python yang digunakan FastAPI sebagai *schema* untuk:

- Memvalidasi data yang masuk dari client (request body)
- Mendefinisikan format data yang dikembalikan ke client (response)
- Auto-generate dokumentasi API di Swagger UI

Contoh: jika client mengirim `price: -500` atau `price: "lima ratus"`, Pydantic langsung menolak dan mengembalikan `422 Unprocessable Entity` dengan pesan error yang jelas — sebelum request bahkan sampai ke database.

8. FastAPI

FastAPI adalah framework Python modern untuk membangun REST API dengan cepat dan mudah. Keunggulannya:

Fitur	Keterangan
Kecepatan	Salah satu framework Python tercepat (setara NodeJS & Go)
Auto-dokumentasi	Swagger UI (/docs) dan ReDoc (/redoc) otomatis ter-generate
Validasi otomatis	Terintegrasi dengan Pydantic untuk validasi request/response
Dependency Injection	Sistem Depends() yang elegan untuk koneksi database, auth, dll.
Type hints	Memanfaatkan type hints Python untuk validasi dan dokumentasi

Panduan Membangun REST API

Dasar Teori

Sebelum memulai implementasi, penting untuk memahami konsep-konsep dasar yang menjadi fondasi dari proyek ini.

1. API (Application Programming Interface)

API adalah "kontrak" atau antarmuka yang mendefinisikan bagaimana dua perangkat lunak berkomunikasi satu sama lain. Dalam konteks web, API memungkinkan frontend (browser/aplikasi mobile) berbicara dengan backend (server) melalui protokol HTTP.

 **Analogi:** API seperti **pelayan di restoran**. Kamu (frontend/client) memesan makanan lewat pelayan (API), pelayan menyampaikan pesanan ke dapur (backend/server), lalu membawa makanan (response) kembali ke mejamu. Kamu tidak perlu tahu cara memasak — cukup tahu cara memesan.

2. REST (Representational State Transfer)

REST adalah gaya arsitektur desain API yang menggunakan HTTP sebagai protokol komunikasi. REST mengorganisasi data sebagai *resources* yang bisa diakses melalui URL yang konsisten dan mudah diprediksi.

Prinsip utama REST:

Prinsip	Penjelasan
Client-Server	Frontend dan backend dipisahkan dan dapat dikembangkan secara independen
Stateless	Setiap request berdiri sendiri — server tidak menyimpan informasi tentang request sebelumnya
Uniform Interface	URL yang konsisten dan dapat diprediksi untuk setiap resource
Resource-Based	Setiap "hal" (item, user, order) adalah sebuah resource dengan URL uniknya sendiri

3. HTTP Methods & CRUD

REST API menggunakan **HTTP Methods** untuk mendefinisikan jenis operasi yang dilakukan pada sebuah resource. Setiap method berkorespondensi dengan satu operasi **CRUD**:

HTTP Method	Operasi CRUD	Contoh Endpoint	Deskripsi
GET	Read	GET /items	Ambil semua items
GET	Read	GET /items/1	Ambil item dengan id=1
POST	Create	POST /items	Buat item baru
PUT	Update	PUT /items/1	Update seluruh data item id=1
DELETE	Delete	DELETE /items/1	Hapus item id=1

4. HTTP Status Codes

Server selalu mengembalikan **status code** di setiap response untuk memberitahu client apakah request berhasil atau gagal dan mengapa.

Kode	Nama	Kapan Digunakan
200	OK	Request berhasil (GET, PUT)
201	Created	Resource baru berhasil dibuat (POST)
204	No Content	Berhasil tetapi tidak ada data dikembalikan (DELETE)
400	Bad Request	Data yang dikirim tidak valid
404	Not Found	Resource tidak ditemukan di server
422	Unprocessable Entity	Validasi gagal — format data salah (default FastAPI)
500	Internal Server Error	Terjadi kesalahan di sisi server

5. Database Relasional & PostgreSQL

Database relasional menyimpan data dalam bentuk **tabel** yang saling terhubung satu sama lain — mirip seperti spreadsheet Excel, tetapi jauh lebih powerful dan handal.

PostgreSQL dipilih dalam proyek ini karena:

- Open-source dan gratis
- Sangat *reliable* dan sudah terbukti di lingkungan produksi
- Mendukung tipe data yang kaya (JSON, Array, UUID, dll.)
- Didukung oleh hampir semua cloud provider (Railway, Render, AWS RDS, Supabase)
- Cocok untuk arsitektur microservices di fase selanjutnya

6. ORM — SQLAlchemy

ORM (Object-Relational Mapping) adalah teknik yang memungkinkan kita berinteraksi dengan database menggunakan objek Python, tanpa harus menulis SQL secara manual.

Perbandingan tanpa ORM vs dengan ORM:

Tanpa ORM (Raw SQL)	Dengan ORM (SQLAlchemy)
<pre>cursor.execute("INSERT INTO items (name, price) VALUES (%s, %s)", ("Laptop", 15000000))</pre>	<pre>db.add(Item(name="Laptop", price=15000000))</pre>
Harus menulis SQL manual	Menggunakan Python object — lebih intuitif
Rentan SQL Injection jika tidak hati-hati	Aman dari SQL Injection secara default
Tidak portable antar database	Bisa pindah database tanpa mengubah kode

7. Pydantic — Validasi Data

Pydantic adalah library validasi data Python yang digunakan FastAPI sebagai *schema* untuk:

- Memvalidasi data yang masuk dari client (request body)
- Mendefinisikan format data yang dikembalikan ke client (response)
- Auto-generate dokumentasi API di Swagger UI

Contoh: jika client mengirim `price: -500` atau `price: "lima ratus"`, Pydantic langsung menolak dan mengembalikan `422 Unprocessable Entity` dengan pesan error yang jelas — sebelum request bahkan sampai ke database.

8. FastAPI

FastAPI adalah framework Python modern untuk membangun REST API dengan cepat dan mudah. Keunggulannya:

Fitur	Keterangan
Kecepatan	Salah satu framework Python tercepat (setara NodeJS & Go)
Auto-dokumentasi	Swagger UI (<code>/docs</code>) dan ReDoc (<code>/redoc</code>) otomatis ter-generate
Validasi otomatis	Terintegrasi dengan Pydantic untuk validasi request/response
Dependency Injection	Sistem <code>Depends()</code> yang elegan untuk koneksi database, auth, dll.
Type hints	Memanfaatkan type hints Python untuk validasi dan dokumentasi

Panduan Membangun Frontend React

Dasar Teori

1. React

React adalah library JavaScript untuk membangun tampilan antarmuka pengguna (UI). React bekerja dengan cara memecah halaman menjadi bagian-bagian kecil yang disebut **komponen**. Setiap komponen memiliki tugas dan tanggung jawabnya masing-masing dan dapat digunakan ulang di berbagai tempat.

💡 **Analogi:** Halaman web seperti sebuah bangunan. Komponen adalah bagian-bagian bangunan tersebut terdapat bagian atap, dinding, pintu, dan jendela. Jika pintu perlu diganti, cukup ganti bagian pintu saja tanpa harus merombak seluruh bangunan.

Konsep utama React yang digunakan dalam proyek ini:

Konsep	Penjelasan	Digunakan di
Component	Fungsi JavaScript yang menghasilkan tampilan (JSX)	Semua file <code>.jsx</code>
Props	Data yang dikirim dari komponen induk ke komponen anak — bersifat hanya-baca	Semua komponen
State	Data yang dapat berubah di dalam komponen — setiap perubahan memperbarui tampilan	<code>App.jsx</code>
JSX	Sintaks penulisan HTML di dalam file JavaScript	Semua komponen
useState	Hook untuk menyimpan dan mengubah data di dalam komponen	<code>App.jsx</code> , <code>ItemForm.jsx</code> , <code>SearchBar.jsx</code>
useEffect	Hook untuk menjalankan kode tertentu saat komponen pertama dimuat atau saat data tertentu berubah	<code>App.jsx</code> , <code>ItemForm.jsx</code>
useCallback	Hook untuk mencegah sebuah fungsi dibuat ulang setiap kali komponen dirender	<code>App.jsx</code>

2. Props dan State

Aspek	Props	State
Sumber data	Dikirim dari komponen induk	Dibuat dan dikelola di dalam komponen itu sendiri
Dapat diubah?	✗ Tidak dapat diubah oleh komponen penerima	☑ Dapat diubah menggunakan fungsi dari <code>useState</code>
Efek perubahan	Jika induk memperbarui props, komponen anak ikut diperbarui	Jika state berubah, tampilan komponen tersebut diperbarui
Analogi	Instruksi yang diberikan atasan kepada karyawan	Catatan kerja milik karyawan itu sendiri

3. Fetch API

Fetch API adalah fitur bawaan JavaScript untuk mengirim permintaan HTTP ke server. Tidak diperlukan instalasi library tambahan untuk menggunakannya.

Jenis permintaan yang digunakan dalam proyek ini:

Tujuan	Method HTTP	Contoh penggunaan
Mengambil semua item	GET	<code>await fetch("/items")</code>
Menambah item baru	POST	<code>fetch("/items", { method: "POST", body: ... })</code>
Memperbarui item	PUT	<code>fetch("/items/8", { method: "PUT", body: ... })</code>
Menghapus item	DELETE	<code>fetch("/items/8", { method: "DELETE" })</code>
Memeriksa status backend	GET	<code>await fetch("/health")</code>

Alasan semua fungsi fetch dipusatkan di `api.js`:

Jika alamat backend berubah di kemudian hari (misalnya saat deployment ke cloud), perubahan hanya perlu dilakukan di satu tempat, yaitu di `api.js`. Tidak perlu mengubah kode di setiap komponen secara satu per satu.

4. Separation of Concerns pada Frontend

Prinsip **Separation of Concerns** berarti setiap file hanya memiliki satu tanggung jawab yang spesifik. Prinsip yang sama diterapkan pada backend (pemisahan `main.py`, `crud.py`, `models.py`) juga berlaku pada frontend.

Struktur folder: `src/components/` — Komponen reusable

File	Tanggung Jawab
<code>App.jsx</code>	Root component, routing, state management user, loading, dan koneksi API
<code>LoginChoicePage.jsx</code>	Halaman untuk memilih jenis akun (Customer/Admin) sebelum login
<code>LoginPage.jsx</code>	Formulir login dengan validasi email dan password
<code>RegisterPage.jsx</code>	Formulir registrasi akun baru untuk customer
<code>ProtectedRoute.jsx</code>	Route guard untuk halaman yang memerlukan autentikasi & role-based access
<code>Header.jsx</code>	Header navigasi, judul aplikasi, link ke halaman, icon cart, dan menu profil
<code>CustomerNav.jsx</code>	Navigasi sidebar/top-nav khusus untuk halaman customer
<code>ItemForm.jsx</code>	Form CRUD produk (mode create & edit) dengan validasi
<code>ItemList.jsx</code>	Container grid untuk menampilkan daftar produk dengan status loading/kosong
<code>ItemCard.jsx</code>	Card produk individual dengan harga, deskripsi, stok, dan tombol aksi
<code>SearchBar.jsx</code>	Kolom pencarian produk dengan input dan tombol clear

Struktur folder: `src/pages/` — Halaman (page components)

File	Tanggung Jawab
Customer Pages	
<code>CustomerHome.jsx</code>	Beranda customer dengan produk featured & call-to-action

File	Tanggung Jawab
ShopPage.jsx	Katalog produk dengan filter, pencarian, dan pagination
ProductDetailPage.jsx	Halaman detail produk dengan deskripsi lengkap, stok, testimoni, dan tombol add to cart
CartPage.jsx	Keranjang belanja dengan list items, update quantity, dan subtotal
CheckoutPage.jsx	Proses checkout & fill data pengiriman sebelum pembayaran
OrdersPage.jsx	Riwayat pesanan customer dengan status, detail, dan timeline pengiriman
TestimoniPage.jsx	Halaman daftar testimonial dari customer lain (publik)
ProfilePage.jsx	Profil customer: data pribadi, riwayat pesanan, dan opsi edit
AboutPage.jsx	Informasi tentang ATHSNAC, visi dan misi, dan contact details
AboutPage.jsx	Aplikasi cloud-native untuk manajemen inventaris
Admin Pages	
AdminDashboard.jsx	Dashboard utama admin dengan KPI, statistik penjualan, dan quick actions
AdminProducts.jsx	Manajemen produk: CRUD, bulk edit, filter, dan statistik inventori
AdminOrders.jsx	Manajemen pesanan: list semua order, update status, dan detail pesanan
AdminPayments.jsx	Verifikasi pembayaran: daftar pembayaran pending, approved, rejected
AdminCustomers.jsx	Data pelanggan: list user, detail profil, dan history transaksi
AdminTestimonials.jsx	Moderasi testimoni: approve/reject/hide/show testimoni dari customer

Struktur folder: **src/services/** — Service layer

File	Tanggung Jawab
api.js	Centralized HTTP wrapper: semua fungsi fetch (GET/POST/PUT/DELETE), token management, error handling

💡 Manfaat pemisahan ini:

- **Maintainability:** Bug di form hanya perlu dibuka `ItemForm.jsx`, bug di halaman list buka `ItemList.jsx/ShopPage.jsx`
- **Reusability:** Komponen `ItemCard.jsx` bisa dipakai di berbagai halaman tanpa duplikasi kode
- **Team Collaboration:** Masing-masing anggota tim bisa mengerjakan file berbeda secara parallel tanpa conflict
- **Testability:** Setiap komponen/halaman bisa di-unit test secara independen
- **Scalability:** Ketika fitur bertambah, mudah menambah file baru tanpa merombak struktur existing

☑ Fitur UI yang Dibangun

Berikut daftar fitur UI berdasarkan halaman dan komponen yang sudah diimplementasikan:

Autentikasi & Security:

Fitur	Komponen	Cara Kerja
Pilih jenis akun	LoginChoicePage.jsx	Switch antara "Masuk sebagai Customer" atau "Masuk sebagai Admin"
Login	LoginPage.jsx	Validasi email & password → login() → POST /auth/login → simpan token ke localStorage
Registrasi	RegisterPage.jsx	Form validasi → register() → POST /auth/register → otomatis login setelah berhasil
Protected routes	ProtectedRoute.jsx	Cek token & role user → redirect ke login jika belum auth atau redirect ke halaman sesuai role
Logout	Header.jsx	Hapus token → clear state user → redirect ke login

Customer Pages:

Fitur	Halaman	Cara Kerja
Beranda	CustomerHome.jsx	Tampilkan hero section, featured products, dan CTA ke shop
Katalog Produk	ShopPage.jsx	GET /products → tampilkan grid produk dengan filter kategori & search
Detail Produk	ProductDetailPage.jsx	GET /products/{id} → tampilkan foto, deskripsi, rating, review
Tambah ke Keranjang	ProductDetailPage.jsx + Header.jsx	POST /cart/items → update cart count di Header
Lihat Keranjang	CartPage.jsx	GET /cart → tampilkan list items dengan quantity, price, total
Update Jumlah	CartPage.jsx	PUT /cart/items/{id} → ubah quantity item
Hapus dari Keranjang	CartPage.jsx	DELETE /cart/items/{id} → update cart view
Checkout	CheckoutPage.jsx	Form alamat pengiriman → POST /orders → create order & redirect ke payment
Riwayat Pesanan	OrdersPage.jsx	GET /orders → list semua order customer dengan status tracking
Testimoni	TestimoniPage.jsx	GET /testimonials (filter is_visible=true) → tampilkan review publik
Buat Testimoni	ProductDetailPage.jsx atau page order detail	POST /testimonials → submit review setelah beli produk

Fitur	Halaman	Cara Kerja
Profil	<code>ProfilePage.jsx</code>	GET /auth/me → tampilkan data user, edit profil, riwayat pesanan

Admin Pages:

Fitur	Halaman	Cara Kerja
Dashboard	<code>AdminDashboard.jsx</code>	GET /products/stats, GET /orders/admin/all → KPI dashboard
Kelola Produk	<code>AdminProducts.jsx</code>	GET /products + POST/PUT/DELETE /products → CRUD produk & update stok
Kelola Pesanan	<code>AdminOrders.jsx</code>	GET /orders/admin/all → PUT /orders/{id} → update status pengiriman
Verifikasi Pembayaran	<code>AdminPayments.jsx</code>	GET /payments → PUT /payments/{id} → approve/reject bukti transfer
Data Pelanggan	<code>AdminCustomers.jsx</code>	GET /users (backend endpoint) → list semua customer & detail transaksi
Moderasi Testimoni	<code>AdminTestimonials.jsx</code>	GET /testimonials (semua) → PUT /testimonials/{id}/toggle-visibility

UI Components:

Fitur	Komponen	Fungsi
Header Navigation	<code>Header.jsx</code>	Judul, navigation links, cart icon with badge, profile dropdown, logout
Sidebar Nav	<code>CustomerNav.jsx</code>	Navigation untuk mobile/desktop di halaman customer
Product Card	<code>ItemCard.jsx</code>	Grid card: foto, nama, harga (format Rupiah), rating, stok, tombol aksi
Search Bar	<code>SearchBar.jsx</code>	Input pencarian realtime + filter kategori + tombol clear
Product Form	<code>ItemForm.jsx</code>	Form create/edit produk dengan validasi: nama, harga, deskripsi, stok, kategori
Empty State	<code>ItemList.jsx</code> atau page components	Tampilan ketika data kosong dengan icon & pesan
Loading State	Semua halaman	Skeleton/spinner saat loading data dari API
Toast Notification	Global (<code>App.jsx</code> & komponen)	<code>react-toastify</code> untuk success/error/info messages
Modal Confirmasi	<code>CartPage.jsx</code> , <code>AdminProducts.jsx</code>	Konfirmasi sebelum delete/checkout

Dokumentasi API

Lihat dokumentasi lengkap di: [docs/api-test-docs.md](#)

Base URL

```
http://localhost:8000
```

Tabel Ringkasan Endpoint

Modul	Method	Endpoint	Deskripsi	Status Sukses	Status Error
SYSTEM	GET	/	Root endpoint (Identitas aplikasi)	200	-
SYSTEM	GET	/health	Health check (Status server & versi)	200	-
SYSTEM	GET	/team	Informasi profil anggota Tim Ignite	200	-
AUTH	POST	/auth/register	Registrasi akun baru (Admin/Customer)	201	400, 422
AUTH	POST	/auth/login	Login untuk mendapatkan JWT Token	200	401
AUTH	GET	/auth/me	Ambil profil user yang sedang login	200	401
PRODUCTS	POST	/products	Buat produk baru (Admin Only)	201	401, 422
PRODUCTS	GET	/products	Daftar produk (+ search & filter)	200	-
PRODUCTS	GET	/products/stats	Statistik inventori & total nilai stok	200	401
PRODUCTS	GET	/products/{id}	Detail produk spesifik	200	404
PRODUCTS	PUT	/products/{id}	Update data/stok produk (Admin Only)	200	404
PRODUCTS	DELETE	/products/{id}	Hapus produk secara permanen	204	404
CART	GET	/cart	Lihat isi keranjang belanja aktif	200	401

Modul	Method	Endpoint	Deskripsi	Status Sukses	Status Error
CART	POST	/cart/items	Tambah produk ke keranjang	201	401, 422
CART	PUT	/cart/items/{id}	Update kuantitas item di keranjang	200	404
CART	DELETE	/cart/items/{id}	Hapus item dari keranjang	204	404
ORDERS	POST	/orders	Checkout/Buat pesanan baru	201	401, 422
ORDERS	GET	/orders	Riwayat pesanan milik customer	200	401
ORDERS	GET	/orders/admin/all	Lihat semua pesanan masuk (Admin Only)	200	403
ORDERS	PUT	/orders/{id}	Update status pengiriman (Admin Only)	200	404
PAYMENTS	POST	/payments	Upload bukti pembayaran	201	401
PAYMENTS	GET	/payments	Daftar pembayaran (Isolasi User/Admin)	200	401
PAYMENTS	PUT	/payments/{id}	Verifikasi pembayaran oleh Admin	200	404
TESTIMONIALS	POST	/testimonials	Buat ulasan produk	201	422
TESTIMONIALS	GET	/testimonials	Daftar testimoni publik	200	-
TESTIMONIALS	PUT	/testimonials/{id}	Update ulasan (Pemilik Only)	200	403, 404
TESTIMONIALS	PUT	/testimonials/{id}/toggle-visibility	Sembunyikan/Tampilkan ulasan (Admin)	200	403

Alur Autentikasi

```

1. REGISTER
  POST /auth/register
  Body: { email, name, password }
    ↓
  Password di-hash dengan bcrypt → disimpan ke database
  Response: data akun (tanpa password) → 201 Created

```

- | |
|--|
| 2. LOGIN
POST /auth/login
Body: { email, password }
↓
Server verifikasi password → buat JWT token
Response: { access_token, token_type, user } → 200 OK |
| 3. AKSES DATA (setiap request ke /items)
Header: Authorization: Bearer <access_token>
↓
Server verifikasi token → proses request → kirim data
Tanpa token → 401 Unauthorized |
| 4. LOGOUT
Token dihapus dari memori browser
Tampilan kembali ke halaman login |

Contoh Response Login

Setelah login berhasil, server mengembalikan data berikut:

```
{
  "access_token": "eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9...",
  "token_type": "bearer",
  "user": {
    "id": 1,
    "email": "10231030@student.itk.ac.id",
    "name": "Desnita Dwi Putri",
    "is_active": true,
    "created_at": "2026-03-17T08:00:00+07:00"
  }
}
```

Nilai `access_token` inilah yang perlu disertakan di setiap permintaan ke endpoint `/items`.

Authentication

ATHSNAC menggunakan **JWT (JSON Web Token)** untuk autentikasi dan otorisasi. Setiap user yang login akan menerima token yang harus disertakan di setiap request yang memerlukan autentikasi.

Konsep JWT

JWT terdiri dari 3 bagian yang dipisahkan oleh titik (.):

```
header.payload.signature
```

- **Header:** Tipe token dan algoritma hashing
- **Payload:** Data user (id, email, role) — dapat di-decode tapi tidak dapat diubah karena signature
- **Signature:** Tanda tangan digital yang membuktikan token valid dan tidak dimanipulasi

1. Register (Daftar Akun Baru)

Endpoint:

```
POST /auth/register
```

Request Body:

```
{
  "email": "user@example.com",
  "name": "Nama Lengkap",
  "password": "secure_password123",
  "role": "customer"
}
```

Response (201 Created):

```
{
  "id": 1,
  "email": "user@example.com",
  "name": "Nama Lengkap",
  "role": "customer",
  "is_active": true,
  "created_at": "2026-04-15T10:30:00+07:00"
}
```

Error Response (422 Unprocessable Entity):

```
{
  "detail": [
    {
      "loc": ["body", "email"],
      "msg": "invalid email format",
      "type": "value_error.email"
    }
  ]
}
```

Validasi:

- Email harus unik (tidak boleh terdaftar sebelumnya)
- Password minimal 8 karakter

- Role harus `customer` atau `admin`

2. Login (Dapatkan Token)

Endpoint:

```
POST /auth/login
```

Request Body:

```
{
  "email": "user@example.com",
  "password": "secure_password123"
}
```

Response (200 OK):

```
{
  "access_token":
  "eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJzdWIiOiJ1c2VyQGV4YW1wbGUuY29tIiwiaWQiOiJEsImVtYWlsIjoidXNlckBleGFtcGx1LmNvbSIsInJvbGUiOiJjdXN0b211ciIsImV4cCI6MTcxNDk0NTcwMH0.9nKu7vQ1pL8mN3oR5sT7uW8xY9zAbCdEfGhIjKlMnOp",
  "token_type": "bearer",
  "user": {
    "id": 1,
    "email": "user@example.com",
    "name": "Nama Lengkap",
    "role": "customer",
    "is_active": true
  }
}
```

Error Response (401 Unauthorized):

```
{
  "detail": "Incorrect email or password"
}
```

Token Properties:

- **Tipe:** Bearer token
- **Durasi:** 60 menit (3600 detik)
- **Format:** JWT dengan 3 bagian (header.payload.signature)

3. Get Current User (Verifikasi Token)

Endpoint:

```
GET /auth/me
```

Header yang Diperlukan:

```
Authorization: Bearer <access_token>
```

Response (200 OK):

```
{
  "id": 1,
  "email": "user@example.com",
  "name": "Nama Lengkap",
  "role": "customer",
  "is_active": true,
  "created_at": "2026-04-15T10:30:00+07:00"
}
```

Error Response (401 Unauthorized):

```
{
  "detail": "Could not validate credentials"
}
```

4. Cara Menggunakan Token di Request

Setiap request ke endpoint yang memerlukan autentikasi harus menyertakan **Authorization** header:

Format:

```
Authorization: Bearer <access_token>
```

Contoh Request (cURL):

```
curl -X GET "http://localhost:8000/cart" \
  -H "Authorization: Bearer eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9..."
```

Contoh Request (JavaScript/Fetch API):


```
const token = localStorage.getItem('access_token');

fetch('http://localhost:8000/cart', {
  method: 'GET',
  headers: {
    'Authorization': `Bearer ${token}`,
    'Content-Type': 'application/json'
  }
})
.then(response => response.json())
.then(data => console.log(data));
```

Contoh Request (Python/Requests):

```
import requests

token = "eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9..."
headers = {
  'Authorization': f'Bearer {token}',
  'Content-Type': 'application/json'
}

response = requests.get('http://localhost:8000/cart', headers=headers)
print(response.json())
```

5. Role-Based Access Control (RBAC)

Beberapa endpoint hanya dapat diakses oleh role tertentu:

Endpoint	Role Diizinkan	Deskripsi
POST /products	admin	Hanya admin dapat membuat produk baru
PUT /products/{id}	admin	Hanya admin dapat mengubah produk
DELETE /products/{id}	admin	Hanya admin dapat menghapus produk
GET /products/stats	admin	Hanya admin dapat lihat statistik
GET /orders/admin/all	admin	Hanya admin dapat lihat semua pesanan
PUT /orders/{id}	admin	Hanya admin dapat ubah status pesanan
PUT /payments/{id}	admin	Hanya admin dapat verifikasi pembayaran

Endpoint	Role Diizinkan	Deskripsi
PUT /testimonials/{id}/toggle-visibility	admin	Hanya admin dapat moderasi testimoni

Error Response (403 Forbidden):

```
{
  "detail": "Access denied. Admin role required."
}
```

6. Token Security Best Practices

Untuk Frontend (React):

```
// ☒ BENAR: Simpan token di localStorage atau sessionStorage
localStorage.setItem('access_token', response.access_token);

// ☒ SALAH: Jangan hardcode token di kode
const token = "eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9...";
```

Untuk Backend (FastAPI):

```
# ☒ BENAR: Simpan secret key di environment variable
SECRET_KEY = os.getenv("SECRET_KEY")

# ☒ SALAH: Jangan hardcode secret key
SECRET_KEY = "super_secret_key_12345"
```

Kapan Token Expired:

- Token berlaku selama **60 menit**
- Setelah token expired, user harus login ulang
- Response server akan mengembalikan **401 Unauthorized**

```
{
  "detail": "Token has expired"
}
```

Dokumentasi

Seluruh dokumen hasil pengujian dan referensi proyek tersedia di folder **docs/**:

Quality Assurance & Testing

File	Status	Keterangan
docs/auth-test-results.md	☒ 19/19 Tests	Hasil pengujian autentikasi & otentikasi JWT — 100% Success Rate
docs/api-test-results.md	☒	Hasil pengujian endpoint API via Swagger UI
docs/ui-test-results.md	☒	Hasil pengujian 10 test case UI React via browser

Development Documentation

File	Lead	Keterangan
docs/setup-guide.md	DevOps	Panduan setup lengkap dari clone hingga running
docs/database-schema.md	DevOps	Skema tabel database PostgreSQL dengan ERD
docs/docker-architecture.md	DevOps	Arsitektur Docker & Docker Compose
docs/api-documentation.md	Backend	Dokumentasi lengkap API endpoints

Roadmap

Minggu	Target	Status
1	Setup & Hello World	☒
2	REST API + Database	☒
3	React Frontend	☒
4	Full-Stack Integration	☒
5-7	Docker & Compose	☒
8	UTS Demo	☐
9-11	CI/CD Pipeline	☐
12-14	Microservices	☐
15-16	Final & UAS	☐